

Problem 4 – Smart Password Generator 🗝️

Difficulty

★ Beginner

Overview

A small software company wants a simple application that helps employees generate strong passwords for their online accounts. The application should allow users to choose the password length and the types of characters they want to include.

Your task is to build a Smart Password Generator that creates secure and random passwords.

Mission (Python)

1. Store the available character groups.

Use the **string** module to access the following groups:

Character Group	Module
Lowercase Letters	<code>string.ascii_lowercase</code>
Uppercase Letters	<code>string.ascii_uppercase</code>
Digits	<code>string.digits</code>
Special Characters	<code>string.punctuation</code>

2. Make the application run forever.

For every user, ask for:

- Desired password length.
 - Include uppercase letters? (Yes/No)
 - Include lowercase letters? (Yes/No)
 - Include digits? (Yes/No)
 - Include special characters? (Yes/No)
-

3. Validate the user's input.

Ensure that:

- Password length is at least **4** characters.
- The user selects **at least one** character group.

If the input is invalid, ask the user to try again.

4. Generate the password.

Use:

- `random.choice()`
- A loop
- The selected character groups

Generate a random password with the requested length.

Bonus Mission ★

Evaluate the password strength.

Length	Strength
--------	----------

4–7	Weak
-----	------

8–11	Medium
------	--------

12 or more	Strong
------------	--------

Display the strength after generating the password.

Useful Hints

- Import:
 - random
 - string
 - Use random.choice().
 - Store the selected characters in one variable.
 - Use a for loop to build the password.
 - Use functions to organize your code.
 - Use while True to repeat the application.
-

GUI Mission (Streamlit)

Create a graphical version of the Password Generator.

Required Text Functions

Use:

- `st.title()`
 - `st.header()`
 - `st.subheader()`
 - `st.write()`
 - `st.divider()`
-

Required Input Widgets

Use:

- `st.slider()` → Password length
- `st.selectbox()` → Include uppercase?
- `st.selectbox()` → Include lowercase?
- `st.selectbox()` → Include digits?
- `st.selectbox()` → Include special characters?
- `st.button()` → Generate Password

Note: Use only the widgets covered in your syllabus. (Checkboxes are intentionally not used.)

Required Status Elements

Use **at least three** of the following:

- `st.info()` → Explain password security tips.
 - `st.success()` → Password generated successfully.
 - `st.warning()` → Password is weak.
 - `st.error()` → No character group selected.
-

Required Layout

Your application **must** include:

Sidebar

Display:

- Password Tips
 - Character Groups
-

Two Columns

Left Column

- Password options
- Generate button

Right Column

Display:

- Generated password
 - Password length
 - Password strength
-

Expected GUI

 Smart Password Generator

Sidebar

Password Tips

- Use at least 8 characters
 - Mix letters and digits
 - Include symbols
-

Password Length

[10]

Uppercase

[Yes ▼]

Lowercase

[Yes ▼]

Digits

[Yes ▼]

Special Characters

[No ▼]

[Generate Password]

Generated Password

A8kP3LmQ9X

Strength: Medium

 Password generated successfully!

Challenge Mission ★★

Add a **Password History** feature.

Every generated password should be saved and displayed in a list.

The history should include:

- Password
- Length
- Strength

Example:

Password History

1. A8kP3LmQ9X , 10 , (Medium)

2. @M4xL8!Qa2P# , 12 , (Strong)

3. abc12345 , 8 , (Weak)

Python Topics Covered

- Variables
 - Strings
 - Lists
 - Arithmetic operators
 - Conditional statements (if, elif, else)
 - for loop
 - while loop
 - User-defined functions
 - random.choice()
 - string.ascii_lowercase
 - string.ascii_uppercase
 - string.digits
 - string.punctuation
-

Streamlit Topics Covered

Displaying Text

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.slider()`
- `st.selectbox()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
- `st.columns()`

Learning Objectives

After completing this project, you should be able to:

- Use the random and string modules to generate random data.
- Build strings dynamically using loops.
- Validate user input before processing.
- Classify results using conditional logic.
- Create a user-friendly Streamlit application that allows users to customize password generation and immediately see the result.